

Contents

Self-Calibrating Per-Tenant Relevance Gating with a Conditional Cross-Lingual Fallback for Offline Multilingual RAG	2
Abstract	2
1. Introduction	2
1.1 The brittle global cutoff	3
1.2 What we claim, and what we do not	3
1.3 Contributions	4
1.4 Reproducibility	4
2. Related Work	5
2.1 RAG and adaptive / self-retrieval gating	5
2.2 Hybrid retrieval, fusion, and reranking	5
2.3 Multi-tenant vector databases	6
2.4 Cross-lingual IR and on-device / private RAG	6
2.5 Unsupervised score-distribution thresholding	6
2.6 Arabic NLP and dialect-aware RAG	7
3. Problem Formulation	7
3.1 Multi-tenant offline RAG	7
3.2 The relevance-gate decision	8
3.3 The unbounded reranker score	8
3.4 Why a single global cutoff is wrong	8
4. Method	9
4.1 Self-calibration from score geometry	9
4.2 Conditional cross-lingual fallback	11
4.3 Why the procedure compensates per-language scoring bias	11
4.4 Design choices and what they buy	12
5. System and Implementation	12
5.1 Architecture overview	12
5.2 Hybrid retriever and per-tenant isolation	13
5.3 The relevance gate in the pipeline	14
5.4 The scorer-agnostic ScorerProfile	14
5.5 Offline local LLM and supporting components	14
5.6 Dialect post-processor and study tools	15
5.7 Security and privacy engineering	15
5.8 Request flow	16
6. Evaluation	16
6.1 Setup	16
6.2 Exp1 — Per-tenant cutoffs (reproducing the deployed system)	17
6.3 Exp2 — Labelled gate-decision accuracy	18
6.4 Exp4 — Ablation of alpha	19
7. Discussion, Limitations, and Threats to Validity	20
7.1 What the results do and do not show	20
7.2 Limitations and threats to validity	21
7.3 Ethics, privacy, and fairness	23
8. Conclusion and Future Work	23
Declarations	24
References	24

Self-Calibrating Per-Tenant Relevance Gating with a Conditional Cross-Lingual Fallback for Offline Multilingual RAG

Ayman Kazim Yousef Department of Artificial Intelligence Engineering, AlSafwa University, Karbala, Iraq ORCID: 0009-0006-7409-9367 · ai25009@student.alsafwa.edu.iq

Abstract

A retrieval-augmented generation (RAG) system must decide, after retrieval and reranking, whether any candidate passage is relevant enough to ground an answer; otherwise it should answer from the model’s general knowledge rather than forcing an off-topic document into the prompt. In practice this decision rests on a single, hand-tuned global score cutoff, which is brittle: a reranker’s score distribution shifts with the corpus, with the language (multilingual cross-encoders score equally-relevant Arabic passages lower than English ones), and — in a multi-tenant system where every user owns a different private library — with the tenant. We present an **unsupervised, label-free, per-tenant self-calibrating relevance gate** for the post-reranking stage of an offline, on-device, multilingual (Arabic+English) digital-library RAG system. The threshold is derived entirely from the relevance scorer’s own score geometry — an in-domain self-match anchor versus a cross-document out-domain anchor — so it requires no user feedback, training, reward, or labeled evaluation set, works on a brand-new (cold-start) tenant, and is computed on-device from local data only. A conditional cross-lingual fallback then translates the query exactly once, only on a calibrated same-language miss. In a deterministic micro-benchmark on the real 3,739-chunk, 10-book corpus run on CPU, the label-free gate reaches F1 0.968 versus 0.828 for the deployable fixed default, raising precision from 0.71 to 0.94 while keeping recall at 1.0; it also automatically derives a higher cutoff for the pure-Arabic tenant in both score spaces — a real but partial, scorer-dependent compensation of the reranker’s per-language score offset (the pure-Arabic tenant is stricter in both score spaces, while the mixed Arabic+English tenant rank-flips across scorers, as we detail in the evaluation). We are explicit about scope: this is a small, English-dominated, passage-span benchmark on one machine, not a large labeled IR evaluation, and we do not yet measure downstream answer quality. The system and experiment scripts are released for reproduction.

1. Introduction

A retrieval-augmented generation (RAG) pipeline retrieves candidate passages, optionally reranks them with a cross-encoder, and feeds the top passages to a language model that generates a grounded answer [14, 26]. Before generation, the pipeline faces a deceptively simple binary decision that we call the **relevance gate**: is any retrieved passage actually on-topic for this query? If yes, the answer should be grounded in that passage; if no, the system should answer from the model’s general knowledge — or trigger a fallback — rather than splice an unrelated document into the prompt, which is a direct cause of wrong, hallucinated, or source-mislabeled answers. Recent adaptive-RAG work makes versions of this decision: Self-RAG learns reflection tokens that critique retrieved relevance [6], Adaptive-RAG routes queries by predicted complexity [20], FLARE triggers retrieval from generation confidence [21], and CRAG runs a lightweight evaluator that classifies retrieval quality and can fall back to web search [46]. Each of these, however, learns its decision policy from supervision, an auxiliary head, or external resources.

1.1 The brittle global cutoff

Operationally, the relevance gate reduces to a **score cutoff** on the reranker’s output: accept the answer-from-passage path iff the best reranked score is at least some threshold τ . In production RAG, τ is almost always a single hand-tuned global constant. This is brittle for three compounding reasons. First, cross-encoder relevance scores are unbounded logits that are not robustly calibrated across inputs [32, 37], and their distribution shifts with the **corpus** — its topic mix and document lengths. Second, the distribution shifts with **language**: a multilingual reranker trained on data such as mMARCO [8] systematically assigns lower scores to equally-relevant Arabic passages than to English ones, so a cutoff tuned on English content silently over-filters Arabic. Third, and most acutely, the distribution shifts with the **tenant**. In a multi-tenant system each user owns a *different* private library; one global cutoff over-filters users whose collections score high and under-filters users whose collections score low. The classical remedy — score-distributional threshold optimization that models the relevant/non-relevant score densities of a ranked list [4, 5, 30], in the spirit of nonparametric histogram thresholding [34] — is exactly the right framing, but the standard variants presume per-query relevant/non-relevant samples that a private, cold-start, on-device deployment does not have.

Multi-tenant vector databases such as Curator [22], and filtered-ANN methods including Filtered-DiskANN [15] and ACORN [36], solve the orthogonal problem of *which* vectors a tenant may search; they isolate retrieval but still leave the relevance cutoff a static, shared constant. On-device and edge RAG systems — MeMemo [44], MobileRAG [35], and EdgeRAG [40] — establish that fully-local, privacy-preserving retrieval is practical, and some prune weak candidates, but none calibrate the cutoff per tenant from local data. The closest prior art is training-free adaptive gating: TARG decides *whether to retrieve* from an unsupervised uncertainty threshold without labels or auxiliary heads [42]. We share its label-free, training-free motivation but move the decision to a different point in the pipeline — the **post-retrieval** relevance cutoff — and make it **per tenant**.

1.2 What we claim, and what we do not

We describe **maktaba-web-local** (al-Maktaba al-Natiga, “The Speaking Library”), a fully offline, on-device, multilingual (Arabic+English) personal digital-library RAG system, and within it a self-calibrating relevance gate. The threshold is computed, per tenant, from the relevance scorer’s **own score geometry**: from a tenant’s private corpus we form a small set of in-domain self-match scores (a passage’s own leading words scored against itself — a guaranteed-relevant upper anchor) and a set of out-domain cross-book scores (those words scored against a chunk of a *different* book — an irrelevant lower anchor), and place τ a recall-favouring fraction of the way into the gap between the irrelevant and relevant bands. No part of this uses user feedback, clicks, training, a reward signal, or a labeled evaluation set; it runs on a brand-new tenant with zero usage history; and it is computed entirely on-device. The gate then conditions a cross-lingual fallback: retrieval and reranking proceed in the query’s language, and only on a calibrated same-language miss is the query translated once (cached, timeout-bounded) and retrieval retried — so translation cost, prohibitive on every query for an edge device, is paid only when genuinely needed [2, 31].

We are deliberate about positioning. We claim a *label-free, per-tenant, cold-start, on-device* calibration procedure that, in a setting where a labeled oracle is unobtainable, attains near-oracle gate accuracy without any labels — and, as an unexpected by-product supporting non-obviousness, partially compensates the reranker’s per-language scoring bias by deriving a higher cutoff for the pure-Arabic tenant in both score spaces (the effect is partial and scorer-dependent — the mixed Arabic+English tenant rank-flips across scorers, as detailed in the evaluation). We do **not** claim to beat a label-tuned cutoff: on our small corpus a global cutoff *chosen with labels* is in fact perfect, but obtaining it requires precisely the labeled evaluation

set that a private, multi-tenant, cold-start deployment lacks. The fair, deployable comparison is therefore label-free self-calibration against the system’s previous hand-tuned global default, and on that comparison the method raises F1 from 0.828 to **0.968**, lifting precision from **0.71 to 0.94** — removing the off-topic acceptances that we expect to produce hallucinated and mis-attributed answers — while keeping recall at 1.0. We also do **not** claim a large-scale or end-to-end result: our evaluation is a reproducible micro-benchmark on one CPU machine over a small (10-book, English-dominated) corpus, not a large labeled IR benchmark; our positive (“relevant”) queries are passage-derived spans rather than LLM-generated natural questions, which likely makes the in-domain class easy and inflates recall; and we have not yet run an end-to-end answer-quality or hallucination-rate user study, so the gate’s downstream benefit is argued from precision, not measured on generated answers. We state these threats to validity in full in the evaluation.

1.3 Contributions

- **A self-calibrating per-tenant relevance gate.** An unsupervised, label-free threshold for the post-retrieval reranking stage that, to our knowledge, is the first to combine all of: (i) being the *post-retrieval* relevance cutoff, (ii) calibrated *per tenant*, (iii) *label-free* (no feedback, clicks, training, evaluation set, or reward), (iv) derived from the relevance scorer’s *own score geometry* (an in-domain self-match anchor versus a cross-document out-domain anchor), (v) *cold-start* (works on a brand-new tenant with zero history), and (vi) computed *on-device* from local data only.
- **A scorer-agnostic construction.** The same procedure calibrates an unbounded cross-encoder logit and a bounded embedding cosine, with appropriate clamping; we report per-tenant cutoffs in both score spaces, and the pure-Arabic tenant receives a higher cutoff in both spaces (while the Arabic-containing tenants are the two strictest in the cross-encoder space and the mixed tenant rank-flips between scorers), so the method partially — and scorer-dependently — compensates the scorer’s per-language bias.
- **A conditional cross-lingual fallback gated on the threshold.** Translation is triggered only on a calibrated same-language miss, retried once, cached and hard-timeout-bounded, so its cost is never paid on a query that is already answerable in-language — a fit for offline edge deployment.
- **A live, deterministic evaluation with honest scope.** On the real 3,739-chunk, 10-book Arabic+English corpus on CPU, we show per-tenant cutoffs spanning a range no single global cutoff can fit; label-free gate F1 0.968 versus a deployable fixed-default 0.828 (precision 0.71→0.94, recall 1.0); near-oracle performance without labels (and a label-tuned oracle that is unobtainable in deployment); and an α -ablation in which the method is robust for $\alpha \geq 0.25$ and at $\alpha = 0.5$ matches the per-tenant oracle.
- **A reduced-to-practice, released artifact.** The complete system — a FastAPI backend with hybrid BM25 (bm25s) + FAISS-HNSW retrieval fused by Reciprocal Rank Fusion, cross-encoder reranking, the gate, a local LLM, RAPTOR and HyDE retrieval, and strict per-user isolation — together with the experiment scripts, is released for reproduction.

1.4 Reproducibility

Every number in this paper is produced by deterministic scripts run live by the author on the real corpus (3,739 chunks across 10 books, English and Arabic), on CPU with the cached models, with no randomness in pseudo-query sampling (queries are taken as fixed leading-word spans, not sampled with an RNG). The retrieval stack uses the bm25s lexical index [28], a FAISS-HNSW dense index [23, 29] over paraphrase-multilingual-MiniLM-L12-v2 embeddings [43], Reciprocal Rank Fusion [10] of the BM25 [38] and dense rankings, and the mmarco-mMiniLMv2-L12-H384-v1 cross-encoder reranker as the relevance scorer M. We release the maktaba-web-local system and the experiment scripts (exp_calibration.py, exp2_gate_accuracy.py, exp4_ablation.py) so the cutoffs, gate-accuracy figures, and ablation can be regenerated

end-to-end; because the procedure is label-free and deterministic, a reader pointing it at the same corpus obtains the same thresholds.

2. Related Work

Our contribution sits at the intersection of five threads: adaptive retrieval gating, hybrid retrieval and reranking, multi-tenant vector indexing, cross-lingual and on-device RAG, and unsupervised score-distribution thresholding, with a sixth grounding in Arabic NLP and dialect-aware RAG. We position our work against each in turn, stating precisely where prior art stops and our per-tenant, label-free relevance cutoff begins.

2.1 RAG and adaptive / self-retrieval gating

Retrieval-augmented generation grounds a language model in retrieved passages [26], and a large body of recent work makes the *retrieval decision itself* adaptive [14]. Self-RAG trains a model to emit reflection tokens that decide, on demand, whether to retrieve and whether a retrieved passage is relevant [6]. Adaptive-RAG routes a query to no-retrieval, single-step, or multi-step retrieval according to a trained question-complexity classifier [20]. FLARE triggers retrieval mid-generation when the model emits low-probability tokens, i.e. from generation confidence [21]. CRAG runs a lightweight retrieval evaluator over retrieved documents and selects a corrective action (correct / ambiguous / incorrect) with a web-search fallback [46]. Most directly, TARG performs *training-free* adaptive gating, thresholding generation uncertainty to decide when to retrieve without any labels or auxiliary heads [42].

These methods are the closest in spirit to ours, and we share their unsupervised, deployment-oriented motivation, especially TARG’s training-free stance. The decisive distinction is *what* is being gated and *how the threshold is set*. Self-RAG, Adaptive-RAG, FLARE, and TARG decide **whether to retrieve** (or how much), and they do so from **LLM-side signals** — reflection tokens, a complexity classifier, token probabilities, or generation uncertainty — against thresholds that are either trained or fixed *globally* for the whole system. CRAG’s evaluator is the nearest analogue to a post-retrieval relevance judge, but its evaluator is a learned component with a single decision policy, not a cutoff that is recalibrated for each tenant. In contrast, our gate operates **after** retrieval and reranking, on the **relevance scorer’s own output**, and sets the **accept/reject cutoff per tenant with no labels, no feedback, no training, and no evaluation set**. None of these systems adapts its relevance threshold to the corpus owned by an individual user, nor derives that threshold from the score geometry of the scorer itself; that per-tenant, label-free, cold-start calibration is what we add. We also inherit a conditional fallback idea analogous to CRAG’s web search, but ours is a *cross-lingual* retry that is triggered only by a calibrated same-language miss, rather than a fixed corrective rule.

2.2 Hybrid retrieval, fusion, and reranking

Our retrieval stack is a conventional, well-understood pipeline; the gate sits on top of it rather than replacing any of it. The lexical half is BM25 [38], implemented with the eager-sparse-scoring bm25s library [28]. The dense half follows learned dual-encoder retrieval [24] and contrastive, label-free dense encoders [19], indexed for approximate nearest-neighbour search with HNSW graphs [29] via FAISS [23]. The two ranked lists are combined with Reciprocal Rank Fusion, a rank-based, parameter-light fusion that needs no score normalization [10]. The fused candidates are reranked by a cross-encoder that scores a [query, passage] pair jointly, in the monoBERT style [32]; the spectrum of neural rankers from late-interaction ColBERT [25] to compact distilled cross-encoders motivates our choice of a MiniLM-based reranker [43] fine-tuned on multilingual MS MARCO [8], which is what makes joint reranking feasible offline. Multiple retrieved passages are then fused at generation time in the fusion-in-decoder tradition [18]. The relevant point for this paper is that all of these components produce *uncalibrated*

scores — BM25 weights are unbounded [38] and cross-encoder logits are likewise unbounded and not directly comparable across queries or corpora — which is precisely why a separate, calibrated relevance cutoff is needed downstream, and why a single global cutoff transfers poorly.

2.3 Multi-tenant vector databases

A second line of work makes retrieval *multi-tenant*: a single shared index serves many users while restricting each query to that user’s own vectors. Curator provides efficient per-tenant indexing for shared vector databases [22]; Filtered-DiskANN augments graph-based ANN with label/attribute filters so traversal stays within vectors matching a predicate [15]; and ACORN supports predicate-agnostic, high-cardinality filtering while retaining ANN performance [36]. Our system likewise enforces strict per-user isolation, filtering by `user_id` at every stage. These systems answer the question *which vectors are searched* for a tenant — they isolate the candidate set — but they say nothing about *how relevant is relevant enough* for that tenant. The threshold that separates a passage worth grounding on from an off-topic one is orthogonal to which vectors are visible, and it differs across tenants even when isolation is perfect. Our per-tenant cutoff is exactly that missing piece: it calibrates the accept/reject boundary per tenant, complementing — not duplicating — per-tenant index isolation.

2.4 Cross-lingual IR and on-device / private RAG

Cross-lingual information retrieval addresses the query/document language mismatch and its translation strategies [31]. Reproducible multi-stage CLIR baselines establish the retrieve-then-rerank recipe across languages [27], and zero-shot cross-lingual reranking with LLMs is shown to be most effective in-language and only competitive cross-lingually for capable multilingual models [2] — evidence that argues for *gating* a cross-lingual step behind a relevance signal rather than always translating. HyDE further demonstrates label-free zero-shot retrieval with cross-lingual transfer via generated hypothetical documents [13]. On the deployment side, a growing on-device, privacy-preserving RAG literature builds browser- or device-local vector stores with no backend: MeMemo runs an in-browser HNSW store for private, personalized generation [44]; MobileRAG and its on-device components target fast, memory- and energy-efficient retrieval on phones with partial-index loading and selective content reduction [35]; and EdgeRAG indexes online for memory-constrained edge devices via embedding pruning and adaptive caching [40]. Our system shares this offline, on-device, per-tenant-private setting. The gap we address is that the relevance filtering in these systems is governed by *static* thresholds or fixed pruning rules chosen at design time; none calibrates its cutoff to the individual on-device corpus, and none couples a cross-lingual fallback to a per-tenant calibrated miss. Our gate computes the cutoff on-device from local data only, and translation cost is paid only on a genuine same-language miss.

2.5 Unsupervised score-distribution thresholding

The closest theoretical neighbours come from classical IR, where the score distribution of a ranked list is modelled to choose a cutoff *without labels*. Manmatha et al. [30] model relevant versus non-relevant scores as a normal-exponential mixture to combine search-engine outputs; Arampatzis et al. [4] optimize where to stop reading a ranked list from truncated score distributions; and Arampatzis & Robertson [5] survey the score-distribution models and their support/convexity pitfalls. Otsu’s nonparametric histogram threshold, which maximizes between-class variance, is a label-free way to split a one-dimensional score histogram into two classes [34]. Separately, modern neural rerankers are known to be poorly calibrated, motivating uncertainty-aware ranking [37], which is exactly why raw cross-encoder logits cannot simply be thresholded with a fixed constant. We build on this tradition but differ in three concrete ways. First, we apply the thresholding **per tenant**, recalibrating for each user’s

corpus rather than fitting one distribution per query or per system. Second, we obtain the two score populations not from a labeled relevant/non-relevant split but from a **self-supervised anchor pair** — an in-domain band of self-match scores (a query against its own source chunk, a guaranteed-relevant upper bound) and an out-domain band of cross-book scores (a query against a chunk from a different book, treated as irrelevant) — so no relevance judgements are required. Third, the whole estimation runs **on-device, cold-start**, on a brand-new tenant with zero usage history, using only the local corpus. The classical methods supply the statistical intuition (relevant and non-relevant scores occupy separable bands); our anchor construction is what makes that intuition operational in a label-free, multi-tenant, private deployment. We note in passing that this filtering also relates to a fairness concern: injecting weakly-relevant retrieved context can degrade RAG fairness, especially for the small local LLMs we target [17, 47], and metamorphic fairness testing reveals such bias under small input perturbations [11]; a post-retrieval relevance filter is one mechanism that prunes such context — though we measure precision, not fairness, and do not claim a fairness result.

2.6 Arabic NLP and dialect-aware RAG

Finally, the multilingual (Arabic + English) and Iraqi-dialect aspects of the system rest on Arabic NLP infrastructure. AraBERT provides a canonical Arabic transformer encoder [3] of the kind that underpins the Arabic side of multilingual rerankers; CAMEL Tools offers an open-source, offline-friendly Python stack for Arabic preprocessing, morphology, and dialect identification [33]; and the NADI shared task establishes nuanced, country- and province-level Arabic dialect identification, including Iraqi, motivating dialect-aware handling [1]. Most directly comparable as a *system*, DziriBOT is a RAG-based conversational agent for a low-resource Arabic dialect [7]. Our work is complementary: rather than contributing new Arabic models or a new dialect agent, we observe that a multilingual reranker exhibits a per-language scoring offset, and that our per-tenant calibration *partially compensates* for it — deriving a higher cutoff for the pure-Arabic library in both score spaces, though the effect is partial and scorer-dependent — without any language-specific tuning. To our knowledge, no prior Arabic-dialect RAG system calibrates its relevance cutoff per tenant or per language in this label-free manner.

3. Problem Formulation

3.1 Multi-tenant offline RAG

We consider a retrieval-augmented question-answering (RAG) system [26] that serves many independent users from a single deployment, runs entirely on-device, and exchanges no data with any external service. A *tenant* t is a single user. Tenant t owns a private corpus C_t , partitioned by source document (book) into $\{D_1, \dots, D_K\}$, where each book is ingested, chunked, and indexed only for that tenant. Every retrieval stage filters on t : the per-tenant sparse index (bm25s [28]), the dense ANN index (FAISS-HNSW [23, 29]), the Reciprocal Rank Fusion (RRF) of the two ranked lists [10], and the cross-encoder reranking stage [32]. Tenant isolation in a shared vector store is itself a studied problem [15, 22, 36]; here we take isolation as a deployment invariant and study the decision made *after* retrieval.

The deployment imposes three constraints that jointly rule out the usual recipes for tuning a relevance cutoff.

- **Offline / on-device.** Computation runs on the user’s machine (CPU, cached models). There is no central server that could aggregate cross-tenant data, and no opportunity to call a hosted reranker or a remote labeling pipeline. This is the same setting targeted by recent on-device RAG work [35, 40, 44].
- **Label-free.** A private corpus arrives without relevance judgments. There are no clicks, no thumbs-up/down feedback, no held-out labeled evaluation set, and no reward signal. Any

threshold must be obtained from the unlabeled corpus alone, in the spirit of unsupervised retrieval [16, 19].

- **Cold-start.** A brand-new tenant has zero usage history. The cutoff must be available from the moment the corpus is indexed, before the first query is ever asked.
- **Privacy-preserving.** Because the corpus contents are private, the cutoff must be computed from local data only; nothing about C_t may leave the device, and no global cutoff fit on someone else’s data is acceptable on privacy grounds in addition to the accuracy grounds below.

3.2 The relevance-gate decision

After fusion and reranking, the pipeline holds a candidate passage list ordered by a relevance scorer M . For a query q , let $s^* = \max_c M(q, c)$ be the best (top-1) reranked score over the retrieved candidates c . The system must take a binary decision:

- **accept** — ground the answer in the retrieved passage(s): the library plausibly contains relevant content, so the local LLM (Ollama/llama.cpp, e.g. qwen2.5) answers *from* the passage;
- **reject** — the library does not contain relevant content for q : answer from the model’s general knowledge, or trigger the conditional cross-lingual fallback (Section 4.2).

The gate is accept iff $s^* \geq \tau_t$, where τ_t is the tenant’s relevance threshold. This is a post-retrieval, per-query *retrieval-decision* problem: like Self-RAG [6], Adaptive-RAG [20], FLARE [21], CRAG [46], and the training-free gate TARG [42], the system decides whether retrieved evidence should be used — but it must decide using only the relevance scorer’s own output and no supervision. Setting τ_t too low admits off-topic passages, which is the dangerous failure mode: a weakly-relevant or cross-document passage that passes the gate yields a hallucinated or mis-attributed answer, and feeding low-relevance context into a small local LLM is exactly the regime in which retrieval has been shown to degrade output quality and fairness [17, 47]. Setting τ_t too high rejects genuinely relevant passages and needlessly falls back. The gate is therefore a precision/recall trade-off on the accept decision.

3.3 The unbounded reranker score

The signal $M(q, c)$ is not a calibrated probability. The cross-encoder (an mMARCO-style multilingual MiniLM reranker [3, 8, 43]) emits an **unbounded relevance logit**: feeding the [query, passage] pair jointly through the encoder [25, 32] produces a real-valued score whose magnitude has no fixed meaning across queries or corpora. The same holds, in a bounded but still uncalibrated form, for an embedding model used as the scorer, which yields a cosine similarity in $[-1, 1]$ (the system’s embedding backbone is paraphrase-multilingual-MiniLM-L12-v2). Neural rankers of this kind are known to be poorly calibrated [37], so a raw score cannot be compared to a fixed universal constant to decide relevance. Selecting an operating point on such a score is the classical score-distribution thresholding problem [4, 5, 30], and choosing a cut between two populations of a 1-D score histogram is the same shape of problem as Otsu’s nonparametric histogram threshold [34]. What is new here is doing it per tenant, label-free, at cold start.

3.4 Why a single global cutoff is wrong

A natural baseline is one hand-tuned constant τ for every tenant — indeed the deployed system originally used a fixed cross-encoder default of -5.0 . This is wrong in principle because the relevance scorer’s score distribution shifts along three axes:

1. **Corpus shift.** The reranker score reflects how a passage compares to *the other passages that happen to be in this corpus*. A library on one tight topic produces high scores even

for loosely-related queries (everything looks somewhat on-topic), whereas a broad library produces lower top-1 scores. The “relevant” band moves with the corpus.

2. **Language shift.** The multilingual reranker does not score Arabic and English pairs on the same numeric scale. An Arabic-heavy library systematically separates relevant from irrelevant at a *different* score level than an English-heavy one. A constant cutoff silently applies a different effective operating point to each language.
3. **Tenant shift.** Because tenants differ in topic mix, document granularity, and language balance, each tenant’s score geometry differs. Empirically (Section 6) the per-tenant cross-encoder cutoffs span $[-3.29, -1.39]$ and the cosine cutoffs span $[0.302, 0.409]$; no single value sits inside every tenant’s separating band.

We make explicit what we do **not** claim. We do *not* claim that no fixed cutoff can score well on any particular corpus: a cutoff tuned **with labels** on a small fixed corpus can be excellent on that corpus (Section 6 reports a label-chosen global cutoff that is in fact perfect there). The claim is narrower and is about *obtainability*: a private, cold-start, on-device, multi-tenant deployment cannot fit such a cutoff, because fitting it requires a labeled evaluation set the deployment does not have. The fair comparison is therefore between a *deployable* label-free threshold and the *deployable* fixed default — not between a label-free method and an oracle. The remainder of this paper constructs a threshold that meets all four constraints of Section 3.1 simultaneously.

4. Method

The gate computes, for each tenant t , a scorer-specific threshold τ_t from the tenant’s own unlabeled corpus, using nothing but the relevance scorer’s *own score geometry*. The idea is to bracket the unknown relevance cutoff between two anchors that we can construct without labels: a **self-match** anchor that is guaranteed relevant, and a **cross-book** anchor that is almost surely irrelevant. The threshold is placed in the gap between them.

4.1 Self-calibration from score geometry

Let M be the relevance scorer (a cross-encoder returning an unbounded logit, or an embedding model returning a cosine). The corpus C_t is partitioned into books $\{D_1, \dots, D_K\}$.

Applicability guard. The procedure needs at least two books to form a cross-book pair, and enough chunks to sample from. We require $K \geq 2$ and $|C_t| \geq 6$; otherwise the score geometry is too thin to trust and we fall back to a safe scorer-specific default τ_{default} .

Deterministic pseudo-queries. We sample $S = 8$ pseudo-queries with no randomness, so the threshold is reproducible and cache-stable. For the i -th sampled chunk c_i , lying in book $D_{\{b(i)\}}$, the pseudo-query q_i is the **first 12 words** of c_i . (Selecting c_i deterministically — e.g. evenly across the corpus — keeps τ_t a pure function of the corpus state.)

Two unlabeled anchor sets. From the pseudo-queries we build:

- **In-domain (self-match) set** — a guaranteed-relevant *upper* anchor: $S_{\text{in}} = \{ M(q_i, c_i) : i = 1..S \}$. Here the query is literally the head of c_i , so (q_i, c_i) is the most relevant pair that can exist; its scores lie at the top of the relevant band.
- **Out-domain (cross-book) set** — an almost-surely-irrelevant *lower* anchor: $S_{\text{out}} = \{ M(q_i, c'_i) : i = 1..S \}$, where c'_i is a chunk drawn from a **different** book $D_{\{b'\}}$, $b' \neq b(i)$. A query from one book paired against a passage of an unrelated book is a non-relevant pair; its scores lie in the irrelevant band.

These two sets play the role of the relevant / non-relevant score populations in classical score-distribution thresholding [4, 30], but they are *constructed* from the corpus rather than

read off labels.

Robust band edges. To avoid letting a single outlier pair set the threshold, we summarize each anchor by a quartile rather than an extremum:

```
out_hi = Q75(S_out)      # top of the irrelevant band (75th percentile of cross-
                           book scores)
in_lo  = Q25(S_in)      # bottom of the relevant band (25th percentile of self-
                           match scores)
```

Gap test and placement. If $in_lo > out_hi$, the relevant and irrelevant bands are *separated* by a gap, and we set the threshold a fraction α of the way into that gap, measured up from the irrelevant edge:

$$\tau_t = out_hi + \alpha \cdot (in_lo - out_hi), \quad \alpha = 0.25$$

With $\alpha = 0.25$ the cutoff sits a quarter of the way into the gap, just above the irrelevant band — a deliberately **recall-favouring** choice (Section 4.4 / ablation in Section 6). If there is no separating gap ($in_lo \leq out_hi$), the geometry does not license a confident cut and we revert to τ_{default} .

Clamping. We clamp τ_t to a scorer-specific valid range to keep it inside the plausible operating region: $[-8, 2]$ for the cross-encoder logit, $[0, 0.95]$ for cosine.

Caching and invalidation. τ_t is a deterministic function of (tenant, scorer, corpus-state), so it is cached under that key and reused for every query. Any change to C_t (ingesting, deleting, or re-chunking a book) changes the corpus-state and invalidates the cache, triggering one recomputation. Computing τ_t costs $2S = 16$ reranker calls per (re)calibration — computed once per corpus-state and cached, not per query; we do not report wall-clock latency.

Algorithm 1 — Per-tenant self-calibrating relevance threshold

Input: corpus C_t partitioned into books $\{D_1..D_K\}$; relevance scorer M ;
sample size $S = 8$; gap fraction $\alpha = 0.25$;
clamp $[lo, hi]$; default τ_{default} .
Output: per-tenant threshold τ_t .

```
1: if  $K < 2$  or  $|C_t| < 6$  then
2:   return  $\tau_{\text{default}}$                                 # applicability guard
3:  $S_{\text{in}} \leftarrow \emptyset$ ;  $S_{\text{out}} \leftarrow \emptyset$ 
4: for  $i = 1 \dots S$  do                                  # deterministic, no RNG
5:    $c_i \leftarrow i$ -th deterministically sampled chunk, in book  $D_{\{b(i)\}}$ 
6:    $q_i \leftarrow$  first 12 words of  $c_i$ 
7:    $c'_i \leftarrow$  a chunk of some book  $D_{\{b'\}}$  with  $b' \neq b(i)$  # cross-book
8:    $S_{\text{in}} \leftarrow S_{\text{in}} \cup \{M(q_i, c_i)\}$           # self-match (relevant)
9:    $S_{\text{out}} \leftarrow S_{\text{out}} \cup \{M(q_i, c'_i)\}$       # cross-book (irrelevant)
10:  $out\_hi \leftarrow Q75(S_{\text{out}})$                         # top of irrelevant band
11:  $in\_lo \leftarrow Q25(S_{\text{in}})$                           # bottom of relevant band
12: if  $in\_lo > out\_hi$  then                                # separating gap exists
13:    $\tau_t \leftarrow out\_hi + \alpha \cdot (in\_lo - out\_hi)$ 
14: else
15:    $\tau_t \leftarrow \tau_{\text{default}}$ 
16:  $\tau_t \leftarrow \text{clamp}(\tau_t, lo, hi)$ 
17: cache  $\tau_t$  under key (tenant, scorer, corpus-state)
18: return  $\tau_t$ 
```

At query time the gate is then simply: rerank the retrieved candidates, take $s^* = \max_c M(q, c)$, and **accept** iff $s^* \geq \tau_t$, else **reject** (Section 4.2).

4.2 Conditional cross-lingual fallback

The threshold also gates a cross-lingual retry, so that translation cost is paid only when it is actually warranted. Given a query q in language L :

1. Retrieve and rerank within C_t **in the query’s language** L , and compute $s^* = \max_c M(q, c)$.
2. If $s^* \geq \tau_t$, **accept** and ground the answer in the top passage(s). Done — no translation.
3. If $s^* < \tau_t$ (a genuine *same-language miss*) **and** translation is enabled, translate q **once** into the corpus’s other language, with the translation **cached** and bounded by a **hard timeout**, and retry retrieval+rerank **once** on the translated query.
 - If the retried best score now clears τ_t , accept on the cross-lingual evidence.
 - Otherwise, **reject** and answer from the model’s general knowledge.
4. If translation is disabled, a same-language miss goes straight to general knowledge.

Translation is therefore *conditional*: it is triggered only by a calibrated miss in the original language, at most once, and only when enabled. This mirrors corrective and adaptive RAG, which escalate to an alternative retrieval action only when an evaluator deems the first retrieval insufficient [20, 46]. The fallback is motivated by cross-lingual IR, where in-language reranking is strongest and cross-lingual matching is the costlier fallback path [2, 27, 31], and it is especially relevant for low-resource Arabic/Iraqi-dialect content [1, 7, 33], where a query and the library may not share a language.

4.3 Why the procedure compensates per-language scoring bias

Nothing in Algorithm 1 inspects the language of the corpus, yet the threshold it produces *automatically* adapts to it. The reason is that both anchors are drawn from the **same** corpus and scored by the **same** scorer M , so any systematic offset in how M scores a given language enters S_{in} and S_{out} identically and is absorbed into where the gap sits.

Concretely, suppose M assigns Arabic [query, passage] pairs higher raw scores than English pairs of equal true relevance. We *observe* such a per-language score offset for this multilingual reranker on our corpus (Section 6); to our knowledge this offset has not been separately quantified for this reranker, and we do not import it from prior work — we use Penha & Hauff [37] only for the general point that neural rerankers are imperfectly calibrated, not for the Arabic-specific direction of the offset. Under such an offset, for an Arabic-heavy tenant the self-match scores in S_{in} rise (so in_{lo} rises) and the cross-book scores in S_{out} also rise (so out_{hi} rises); the *entire* score geometry shifts upward together. Because $\tau_t = out_{hi} + \alpha \cdot (in_{lo} - out_{hi})$ is anchored to that tenant’s own band edges, the derived cutoff rises with them. The method thus tends to produce a **higher** cutoff for Arabic-containing libraries and a lower one for purely English ones — without ever being told which is which, and without any language-specific tuning. In our experiments (Section 6) this compensation is real but partial and scorer-dependent: the pure-Arabic tenant earns a higher cutoff in *both* the cross-encoder logit space and the cosine space, and the two Arabic-containing tenants are the two strictest in the cross-encoder space, but the mixed Arabic+English tenant is the cross-encoder maximum yet the cosine minimum — a rank-flip across scorers, so the effect is not a clean scorer-independent “Arabic = strictest” law. This is the sense in which the procedure is *scorer-agnostic*: it never assumes a particular numeric scale, only that relevant self-matches and irrelevant cross-book pairs separate, and it places the cut relative to whatever scale the scorer happens to use for this tenant.

4.4 Design choices and what they buy

- **Quartiles, not extrema.** Using $Q75(S_{out})$ and $Q25(S_{in})$ rather than $\max/\min$ makes the band edges robust to a single anomalous pair, at the cost of mild conservatism in the gap width.
- **$\alpha = 0.25$ is recall-favouring.** Placing the cut just inside the irrelevant edge keeps recall high on the harder, more variable scores of real natural-language queries; the ablation (Section 6) shows recall stays at 1.0 across $\alpha \in [0, 1]$, and that larger α would be expected to trade recall headroom for precision on harder queries (recall is pinned at 1.0 on this easy-IN benchmark, so no recall is actually traded away here). We adopt the conservative 0.25 as the safe default.
- **Default on under-determination.** When the corpus is too small ($K < 2$ or $|C_t| < 6$) or no separating gap exists, the method declines to over-fit a cut and uses a safe default — failing safe rather than emitting a spurious tenant-specific threshold.
- **Cost.** Calibration is $2S = 16$ scorer calls, computed once per corpus-state and cached on-device; no per-query overhead is added beyond the single comparison $s^* \geq \tau_t$.

5. System and Implementation

This section describes `maktaba-web-local` (al-Maktaba al-Natiqa, “The Speaking Library”), the deployed system in which the self-calibrating per-tenant gate of Section 4 is reduced to practice. The system is a fully offline, on-device, multilingual (Arabic + English) personal digital library with retrieval-augmented question answering. The backend is roughly 11k lines of Python on FastAPI, with an optional native C++ component for PDF chunking. The entire stack — ingestion, retrieval, reranking, gating, and generation — runs on a single machine from local data, with no network dependency at query time; this design goal follows the on-device, privacy-preserving RAG line of work exemplified by MeMemo [44], MobileRAG [35], and EdgeRAG [40]. We release the system and the experiment scripts for reproduction.

5.1 Architecture overview

The pipeline has six stages. Documents (books) are ingested once; the remaining stages run per query.

1. **Ingestion.** A PDF is parsed with PyMuPDF, normalised, and split into chunks either by the native C++ `smart_chunk` routine or by LangChain’s `RecursiveCharacterTextSplitter` when the native module is absent. Each chunk is embedded and written to the vector store together with its metadata (`user_id`, `book_title`, `page`, and an optional `raptor_level`). Ingestion runs in a background thread and is tracked in a progress dictionary persisted to an `ingestion_ledger.json`, so an interrupted import resumes rather than duplicating chunks. Optionally, a RAPTOR summary tree [39] is built at ingestion time (cluster leaf chunks $\rightarrow$ summarise $\rightarrow$ recurse), and its summary nodes are stored as ordinary chunks tagged with `raptor_level`, so they are retrieved transparently alongside leaf chunks at query time.
2. **Hybrid retrieval.** For each query the system runs sparse and dense retrieval and fuses their ranked lists (Section 5.2).
3. **Reranking.** The fused candidate pool is rescored jointly by a multilingual cross-encoder (Section 5.3), following the retrieve-then-rerank design of monoBERT [32] and reproducible CLIR baselines [27].
4. **The relevance gate.** The best reranked score is compared against the tenant’s self-calibrated cutoff. Below the cutoff, no passage is grounded and the system either triggers the conditional cross-lingual fallback or answers from the model’s general knowledge (Section 5.4).

5. **Generation.** A local LLM (Section 5.5) produces an answer, grounded in the retrieved passages when the gate accepts.
6. **Dialect post-processing.** The answer is rewritten into Iraqi dialect by a deterministic post-processor (Section 5.6).

Every stage degrades gracefully: a missing optional dependency (native engine, reranker, HyDE, translation LLM) disables a feature but never crashes the request, and the system falls back to a simpler path.

5.2 Hybrid retriever and per-tenant isolation

Retrieval is implemented in `hybrid_retriever.py`. The sparse half is BM25 [38] via the `bm25s` library [28], with `rank_bm25` as a pure-Python fallback. The dense half is approximate nearest-neighbour search over the embedding space of `paraphrase-multilingual-MiniLM-L12-v2` (a distilled MiniLM encoder [43]), motivated by dense retrieval results such as DPR [24]. The two ranked lists, together with any HyDE list [13] and template query variants, are merged by Reciprocal Rank Fusion [10] using the standard constant $k = 60$. The retriever over-fetches widely (six times the requested k) before reranking, so that a small book is not crowded out of the candidate pool by a large one, and only the cross-encoder decides final relevance.

Two vector backends. Vectors are stored in Qdrant in local (embedded) mode by default: `QdrantClient(path=...)` runs the vector engine *inside* the FastAPI process, with no server, no network, and no cloud — the configuration that realises the offline goal. A single OS file lock on the data directory means one process at a time; a stale lock left by a hard kill is cleared before connecting. Setting `QDRANT_URL` switches to server mode against a separately running Qdrant (which may still be entirely local), lifting the single-process constraint when concurrency is needed.

FAISS local mode for latency. Qdrant local mode performs brute-force search (linear in the collection size), which is too slow once a collection holds hundreds of thousands of vectors. The system therefore builds an in-memory FAISS HNSW index [23, 29] over all stored vectors in a background thread at startup, with `IndexHNSWFlat` under inner-product metric on L2-normalised vectors. This index is the fast path for dense search at our scale; the Qdrant brute-force path is used only while the FAISS index is still building. We do not report wall-clock latency. The FAISS index is disk-cached and staleness-checked against the Qdrant vector count, so restarts reload in seconds; after a new book is ingested only that book’s vectors are scrolled and added incrementally rather than rebuilding from scratch.

Strict multi-tenant isolation. Every retrieval stage filters on `user_id`, so a tenant only ever sees its own corpus, in keeping with multi-tenant vector-index work [15, 22, 36]. Dense search over-fetches candidates ($8\times$) and then applies the `user_id` predicate as a post-filter, and Qdrant queries carry an explicit `user_id` field condition. Sparse search, however, needs more than a post-filter. A global BM25 index ranks by corpus-wide term statistics, so when one tenant owns most of the corpus (we observed a 97% / 2% / 1% split live, i.e. 2697 / 58 / 33 chunks across three users), a minority tenant’s chunks are outranked off the top-N and post-filtering returns almost nothing. The retriever therefore builds a **per-user BM25 sub-index**: on the first query for a user, it tokenises that user’s own chunks into a private `bm25s` index, caches it in RAM, and ranks within it. The cache is keyed on the corpus state (`id`, `len`) so it self-invalidates when the corpus changes, is size-bounded across many tenants, and is cleared on rebuild. Dense search escapes only the *corpus-statistics* half of this effect — cosine similarity does not depend on corpus composition — but its over-fetch-then-post-filter path is not itself immune to crowding under tenant dominance (a companion analysis measures the same collapse on the dense side); the per-user sub-index is the structural remedy specifically on the sparse side, where no in-traversal filtered-search alternative exists. This isolation is what makes per-tenant calibration meaningful: each tenant’s score geometry is computed strictly from its own books.

5.3 The relevance gate in the pipeline

The cross-encoder reranker (`reranker.py`) is `cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`, a ~52 MB multilingual model trained on mMARCO [8] whose Arabic competence rests on Arabic transformer encoders [3]. It scores each (query, passage) pair jointly and emits an unbounded relevance logit. The reranker exposes two entry points: `rerank(query, docs, top_k, min_score)`, which orders candidates and drops any pair below the cutoff, and `score_pairs(pairs)`, which returns the raw score *distribution* over a set of pairs and is the hook the gate uses for calibration.

The gate itself lives in `relevance_gate.py`. At retrieval time, the `HybridRetriever._tenant_gate(user_id)` method resolves the tenant’s corpus (the per-user sub-corpus described above), asks the gate for that tenant’s cutoff in the active scorer’s score space, and returns it to the reranking stage. Crucially, the gate is applied *during* reranking, not as a separate post-filter: `rerank` is run whenever candidates exist (not only when there are more than k), so an off-topic query has its weak matches *removed* rather than merely reordered. If no passage clears the cutoff, the reranker returns an empty list, and `get_context` reports “no book context,” at which point the caller triggers the conditional fallback or answers from general knowledge. This places our learned threshold in the same architectural slot as the relevance evaluator of CRAG [46] and the post-retrieval filters of FairFilter [47] and Selective Content Reduction [35], but computed per tenant and without labels.

5.4 The scorer-agnostic ScorerProfile

A cross-encoder needs PyTorch and is the heaviest component in the pipeline; it cannot be assumed present on a low-end student laptop or phone. To keep the gate runnable everywhere, calibration is decoupled from the specific scorer through a small `ScorerProfile` abstraction: a named tuple of a scoring function, a safe default cutoff, and a clamp band `[lo, hi]` appropriate to that score space. The gate’s calibration procedure of Section 4 — sample pseudo-queries, build the in-domain (self-match) and out-domain (cross-book) score sets, place the cutoff a quarter into the separating gap between the out-domain 75th percentile and the in-domain 25th percentile, clamp, and cache per (tenant, scorer, corpus-state) — is written once and runs unchanged for any profile.

Two profiles are provided. The default `XENC_PROFILE` wraps the cross-encoder, scores in the unbounded-logit space, and clamps to `[-8, 2]`. When no reranker is available, the retriever constructs a bi-encoder profile that reuses the embedding model *already loaded for vector search* to score cosine similarity between query and passage, clamped to `[0, 0.95]`; this needs no extra model and no PyTorch. The gate mode is reflected end-to-end: a cross-encoder cutoff (`'xenc'`) is applied at the rerank step, whereas a cosine cutoff (`'bienc'`) is applied at vector-search time and is therefore not double-applied at reranking. Because the identical self-calibration runs in either score space, the method is scorer-agnostic by construction — the property our experiments confirm empirically, with the same procedure yielding sensible cutoffs for both the cross-encoder (span `[-3.29, -1.39]` across five tenants) and the embedding cosine (`0.382, 0.329, 0.409, 0.302, 0.325`), and in both spaces assigning the pure-Arabic tenant a higher cutoff (with the mixed Arabic+English tenant rank-flipping across the two scorers).

5.5 Offline local LLM and supporting components

Generation uses a local LLM through a multi-provider client (`offline_llm.py`) whose default offline chain is `llama.cpp` (a local `.gguf` model) then Ollama (e.g. `qwen2.5`), with optional cloud providers selectable only by explicit configuration. The client streams tokens over server-sent events and strips reasoning (`<think>...`) and stray non-target-language preamble from model output. Two retrieval-quality components also depend on this local LLM and are

therefore strictly optional and timeout-bounded: **HyDE** [13], which generates a hypothetical passage in the query’s language to retrieve in passage space (skipped silently if the LLM is unreachable, hard-bounded so it adds no latency when absent), and the **conditional cross-lingual fallback**, which translates the query once — cached, hard-timeout-bounded — only on a genuine same-language gate miss. This conditional design pays the translation cost solely when the in-language pass found nothing, and is motivated by evidence that reranking is strongest in-language and only sometimes competitive cross-lingually [2, 31]. Embeddings are computed through ONNX Runtime (CPU by default, CUDA when available), keeping the dense encoder fast and dependency-light on commodity hardware in the spirit of edge-RAG systems [40].

5.6 Dialect post-processor and study tools

The final answer is rewritten into Iraqi dialect by a deterministic post-processor (`dialect_processor.py`) built from real Iraqi corpora. It applies, in order, morphological transformations (e.g. future-tense restructuring), a length-sorted word-substitution map, and a conservative spacing fixer, while preserving code blocks verbatim. The processor is deliberately lexicon-cautious: earlier word-splitting heuristics that fired on particle substrings inside valid words (corrupting common vocabulary) were removed, leaving only rules that cannot alter letters inside a word. Dialect-aware handling of this kind is motivated by Arabic dialect identification and tooling [1, 33] and by contemporaneous dialect-RAG systems [7].

On top of the same retrieved content, the system provides study tools whose pedagogical value rests on retrieval practice and spaced practice being among the highest-utility learning techniques [12]. Quiz generation draws a broad, evenly spaced sample of a book’s own chunks — deliberately *bypassing* the relevance gate, which is the wrong tool for whole-book coverage — and prompts the local LLM to produce multiple-choice questions [9], with parsing and validation of the model output. Flashcards are scheduled by a pure, deterministic SM-2 spaced-repetition implementation (`srs.py` [45]), with the contemporary spacing-optimisation literature as further grounding [41]. A notes/highlights store completes the study surface. All study state is per-tenant and stored locally.

5.7 Security and privacy engineering

Privacy is enforced both by architecture and by deployment. Architecturally, all user content is scoped by `user_id` at every layer — vector-store filters, the `books/{user_id}/` file layout, and the chat-session tables — and queries and deletions never cross tenants; this is the same isolation that the gate relies on for per-tenant calibration. Because retrieval, reranking, gating, and (in the default configuration) generation all run on-device from local data, no document text, query, or score ever leaves the machine, which addresses the privacy motivation of on-device RAG directly [44] and sidesteps a class of context-injection fairness concerns raised for RAG pipelines [17].

Authentication (`auth.py`) uses SQLite with bcrypt password hashing (cost 12) and a constant-time dummy-hash path that runs bcrypt even for unknown usernames to prevent timing-based account enumeration. Session tokens are 32-byte random strings stored **SHA-256-hashed at rest**, with the raw token living only in an `HttpOnly` cookie, so a stolen database yields no usable sessions and there are no JWT signing keys to manage; legacy plaintext rows are upgraded in place on first use. The request path is wrapped by an ASGI middleware stack providing security headers (including a content-security policy), defence-in-depth origin checking against cross-origin mutations, request logging, and per-IP rate limiting (static-asset paths excepted so parallel script loads are not throttled). Application guards add per-user chat rate limiting, duplicate-request deduplication, and a circuit breaker around the local LLM, all in dependency-free standard-library code.

5.8 Request flow

A chat request proceeds as follows. The browser opens a streaming connection to the FastAPI backend; the middleware stack and the application guards (rate limit, dedup, circuit breaker) admit the request. Context resolution first checks for small talk (answered directly, no retrieval) and otherwise calls `HybridRetriever.retrieve(query, user_id=...)`. Inside `retrieve`: the query is normalised (Arabic question openers, including Iraqi-dialect openers, are stripped) and expanded into template variants; for each variant, BM25 (over the tenant’s per-user sub-index) and dense FAISS search run, optionally in parallel on the native C++ thread pool, each filtered to the tenant; an optional HyDE list is added under a hard timeout; the lists are fused by RRF; the fused pool is reranked by the cross-encoder, which applies the tenant’s self-calibrated cutoff and returns only passages that clear it. If nothing clears the cutoff and translation is enabled, the query is translated once and retrieval is retried a single time; if still nothing clears, no book context is returned. The surviving passages (RAPTOR summaries first, then leaf chunks) are formatted into a labelled context block and passed, with the system prompt, to the local LLM, which streams its answer. Each streamed answer is rewritten into Iraqi dialect and relayed to the browser. The whole path executes locally; the only optional outbound dependency is a cloud LLM provider, which is off by default.

6. Evaluation

We evaluate the self-calibrating per-tenant relevance gate with a small, fully reproducible micro-benchmark run live by the author on the real deployed corpus. We are deliberately explicit about what this evaluation does and does not establish. It is **not** a large labelled IR benchmark; it is a single-machine, deterministic measurement on a 10-book, English-dominated personal library, designed to answer three concrete questions: (i) does the label-free calibration recover the per-tenant cutoffs the deployed system actually uses, and does it expose the observed per-language score offset (Exp1); (ii) on a labelled accept/reject task, how does the label-free gate compare against the deployable fixed default and against label-tuned oracles that a private, cold-start, on-device deployment cannot obtain (Exp2); and (iii) how sensitive is the gate to the gap-position hyperparameter α (Exp4). All scripts, score dumps, and the corpus manifest are released for reproduction.

6.1 Setup

Corpus. The benchmark runs on the author’s real library: **3,739 chunks across 10 books**, mixed English and Arabic, ingested through the production pipeline (PyMuPDF extraction, chunking, hybrid BM25 + FAISS-HNSW retrieval, cross-encoder reranking). From this library we define **five heterogeneous themed tenants**, each a private sub-corpus over a different subject mix, so that the multi-tenant calibration is exercised on genuinely different score geometries rather than re-partitions of one topic:

Tenant	Theme	Books	Chunks
T1	EN ML/AI-search	3	1,535
T2	EN economics/publishing	2	2,010
T3	Arabic + math	4	177
T4	Mixed AR+EN	4	2,733
T5	STEM/electronics	3	45

These tenants span two languages, two orders of magnitude in tenant size (45 to 2,733 chunks), and topic mixes from pure-English ML to Arabic-heavy mathematics, which is precisely the heterogeneity a single global cutoff must contend with.

Setup caveat: the tenants are overlapping sub-corpora, not independent libraries.

The five tenants are themed *sub-corpora* carved from one shared 10-book library, not five independently-owned libraries, and some of them share books: T4 (Mixed) reuses books from T1, T2, and T3, and T3 and T5 share math books. Consequently the “disjoint-topic” OUT queries (Section 6.1, labelled construction) are only *approximately*, not strictly, book-disjoint from a given tenant’s library. We state this plainly as a limitation of the benchmark construction.

Models. The relevance scorer is the multilingual cross-encoder reranker `mmarco-mMiniLMv2-L12-H384-v1`, which emits unbounded relevance logits. The embedding model is `paraphrase-multilingual-MiniLM-L12-v2`, used both for the FAISS-HNSW dense index and for the cosine-space variant of the gate in Exp1. The cross-encoder is the distilled-MiniLM family of compact rerankers [32, 43] trained on multilingual MS MARCO [8], and the dense index uses HNSW [29] under FAISS [23]; the lexical half uses `bm25s` [28] over the Okapi BM25 weighting [38], fused by Reciprocal Rank Fusion [10].

Hardware and determinism. All experiments run on **CPU** with the models cached locally, matching the offline on-device deployment target [35, 44]. The procedure is **deterministic**: the $S = 8$ pseudo-queries are sampled without any RNG (each query is the first 12 words of a chunk, chosen by position), the quantile-based cutoff is a fixed function of the score sets, and the labelled queries are derived by fixed rules, so re-running the scripts reproduces every number below to floating-point precision.

Labelled accept/reject construction (Exp2, Exp4). To measure gate *decisions* against ground truth, we build a labelled probe set per tenant. **IN queries** (positive class, “should accept and ground”) are 18-word spans of passages drawn from **the tenant’s own library, with the source passage left in the candidate pool** — so each IN query is an *easy near-self-match* against the very passage it was cut from (the source is not held out or excluded). The library does contain relevant content for these. **OUT queries** (negative class, “should reject and answer from general knowledge or trigger fallback”) are 18-word spans from **disjoint-topic libraries** plus a fixed set of general-knowledge questions in both English and Arabic, for which the tenant’s library has no relevant passage. For each query we run BM25 top-k over the tenant’s chunks, rerank with the cross-encoder, and the gate decides on the **best reranked score**; the positive class is *relevant/accept*.

Threat to validity (stated up front). The IN queries are *passage-derived spans*, not LLM-generated natural-language questions, and — crucially — the source passage each span is cut from **remains in the candidate pool**, so the gate is asked to match a span against the very passage it came from. A verbatim span scored against its own source is an easy, near-self-match probe, which is the most likely reason recall is 1.0 throughout (it is no accident that recall is exactly 1.0 across every configuration). We do not hold out the source passage or perform sibling-matching, and we do not claim either; the IN class is therefore easy by construction. The reported accept/reject accuracy should therefore be read as a measurement of whether the *gate threshold* sits in the right place given well-separated score evidence, **not** as an estimate of end-to-end retrieval quality on hard conversational queries. Relatedly, we run **no end-to-end answer-quality or hallucination-rate study**: the downstream benefit we claim for the gate (fewer mis-sourced answers) is argued from *precision* on the accept/reject task, not measured on generated answers. We return to both limitations in the discussion.

6.2 Exp1 — Per-tenant cutoffs (reproducing the deployed system)

Exp1 runs the full label-free calibration of Section 4 on each tenant, in **both** score spaces — the cross-encoder’s unbounded logits and the embedding model’s cosine — and reports the resulting per-tenant threshold `tau_t`. This reproduces the cutoffs the deployed system

computes and caches. In every tenant the in-domain (self-match) and out-domain (cross-book) anchor sets were cleanly separated (a gap existed), so the calibrated branch fired in all cases.

Tenant	Theme	Cross-encoder tau_t	Embedding-cosine tau_t
T1	EN ML/AI-search	-2.93	0.382
T2	EN economics/publishing	-2.75	0.329
T3	Arabic + math	-1.59	0.409
T4	Mixed AR+EN	-1.39	0.302
T5	STEM/electronics	-3.29	0.325

Two findings stand out.

A single global cutoff cannot fit all tenants. The cross-encoder cutoffs span **[-3.29, -1.39]** — a range of roughly 1.9 logits across tenants drawn from the *same* corpus with the *same* scorer. The system’s old hand-tuned global default of -5.0 sits well below the entire span, and (as Exp2 confirms) admits large numbers of off-topic passages; conversely no single value in the observed span is correct for all five tenants simultaneously. The cosine cutoffs show the same spread (0.302 to 0.409), so this is not an artefact of the cross-encoder’s unbounded scale.

Arabic-containing tenants get stricter cutoffs — a real but partial, scorer-dependent per-language compensation. The effect is genuine and interesting, but it is not a clean scorer-independent “Arabic = strictest” law, and we state the true picture precisely. In the **cross-encoder logit space**, the two *Arabic-containing* tenants are the two strictest of all five: T4 (Mixed AR+EN) is the maximum at -1.39 and T3 (Arabic + math) is next at -1.59, both above every English-dominant tenant (-2.75 for T2, -2.93 for T1, -3.29 for T5). In the **cosine space**, the *pure-Arabic* tenant T3 is the strictest (0.409), again the maximum over all five. So the pure-Arabic tenant T3 sits high in *both* score spaces. However, the **Mixed tenant T4 rank-flips across scorers**: it is the cross-encoder *maximum* (-1.39) yet the cosine *minimum* (0.302). The honest reading is therefore that the procedure raises the gate for the pure-Arabic library in both score spaces, and for the Arabic-containing libraries in the cross-encoder space, i.e. a **real but partial, scorer-dependent compensation** of the multilingual scorer’s per-language score offset — not a scorer-independent rule that orders tenants identically in every space. Because the calibration derives tau_t from the scorer’s *own* in-/out-domain score geometry on the tenant’s data, it absorbs whatever per-language offset that *particular* scorer exhibits as a side effect, with no language-specific code, label, or tuning. This is the non-obvious behaviour from the contribution, stated at its true strength: we observe the per-language offset on this corpus (to our knowledge it has not been separately quantified for this reranker), and the calibration compensates it partially and scorer-dependently rather than perfectly. We invoke Penha & Hauff [37] only for the general point that neural rerankers are imperfectly calibrated, not for the Arabic-specific direction of the offset.

Because the same procedure produces sensible, separated, per-tenant cutoffs in two unrelated score spaces (unbounded logits and bounded cosine), the method is **scorer-agnostic**: it needs only that the scorer rank a self-match above a cross-document pair, not any particular calibration or output range.

6.3 Exp2 — Labelled gate-decision accuracy

Exp2 evaluates accept/reject decisions on the labelled IN/OUT probe set (Section 6.1) across all five tenants (60 IN + 60 OUT decisions), comparing four threshold strategies: (1) our **self-calibrated, label-free** per-tenant gate; (2) the system’s old hand-tuned **fixed global default of -5.0**; (3) the **best global fixed cutoff chosen with labels** (an oracle, found to be +3.04); and (4) a **per-tenant oracle** (the best per-tenant cutoff chosen with labels).

Strategy	Uses labels?	Acc	Precision	Recall	F1
(1) Self-calibrated (label-free)	no	0.967	0.938	1.000	0.968
(2) Fixed -5.0 (deployable default)	no	0.792	0.706	1.000	0.828
(3) Best global fixed +3.04 (oracle)	yes	1.000	1.000	1.000	1.000
(4) Per-tenant oracle	yes	0.992	0.984	1.000	0.992

The fair, label-free comparison. Two of these strategies are obtainable in our setting and two are not. A private, cold-start, on-device, multi-tenant deployment has **no labelled evaluation set** — no per-tenant relevance judgements and no held-out accept/reject labels — so strategies (3) and (4), both of which select their cutoff *with labels*, are simply unavailable in practice. The honest comparison is therefore between the two label-free strategies that a real deployment can actually run: the **self-calibrated gate (F1 0.968)** versus the **deployable fixed default (F1 0.828)**. Against that baseline the method improves **precision from 0.706 to 0.938** while holding **recall at 1.000**. Concretely, the fixed -5.0 default accepts 25 off-topic passages (false positives) where the self-calibrated gate accepts only 4 — it removes 21 of the 25 spurious acceptances. Those off-topic acceptances are exactly the cases we expect to produce hallucinated or mis-attributed answers (a passage from an unrelated book is fed to the LLM as if it were the source) — a downstream effect we argue from precision rather than measure, so raising precision is the operationally meaningful gain we can demonstrate.

On the oracles. We do not claim to beat the oracle, and we report its number plainly: the best *global* fixed cutoff chosen with labels (+3.04) is **perfect (F1 1.000)** on this small corpus, and the per-tenant oracle reaches F1 0.992. This is unsurprising — on 10 well-separated books a single label-tuned line can split accept from reject cleanly. The point is not that the gate is more accurate than a label-tuned cutoff; it is that **obtaining that label-tuned cutoff requires a labelled set the target deployment does not and cannot have**. What the method delivers is **near-oracle accept/reject quality (0.968 vs the 0.992 per-tenant oracle) with zero labels, zero user feedback, and zero per-tenant tuning**, on a brand-new tenant at cold start. That is the regime in which the contribution matters, and the regime in which the oracle is a counterfactual rather than a deployable competitor.

6.4 Exp4 — Ablation of alpha

The gate places τ_t a fraction α of the way from the top of the irrelevant band (Q75 of S_{out}) into the separating gap toward the bottom of the relevant band (Q25 of S_{in}). Exp4 sweeps α over the same labelled probe set; the default is $\alpha = 0.25$ (recall-favouring).

alpha	Acc	Precision	Recall	F1
0.00	0.642	0.583	1.000	0.736
0.10	0.817	0.732	1.000	0.845

alpha	Acc	Precision	Recall	F1
0.25 (default)	0.967	0.938	1.000	0.968
0.50	0.992	0.984	1.000	0.992
0.75	0.992	0.984	1.000	0.992
1.00	0.992	0.984	1.000	0.992

Recall is 1.000 at every setting of alpha, because any cutoff inside the separating gap still admits all in-domain passages; alpha only trades off how aggressively the gate excludes the borderline-irrelevant tail, i.e. precision. At alpha = 0 the gate sits at the top of the out-domain band and lets the worst off-topic passages through (F1 0.736); F1 then climbs to 0.968 at alpha = 0.25 and 0.992 at alpha = 0.50.

The plateau at alpha $\geq$ 0.50 is a clamping artifact, not continued gap-position insensitivity. The identical F1 of 0.992 at alpha in {0.50, 0.75, 1.00} should *not* be read as evidence that the cutoff keeps improving and then saturates on its merits. As alpha grows, the gap-placement formula $\tau_t = \text{out_hi} + \alpha(\text{in_lo} - \text{out_hi})$ pushes several tenants' cutoffs against the cross-encoder clamp ceiling $hi = +2.0$: at alpha $\geq$ 0.50 several tenant cutoffs are already clamped to +2.0, and at alpha $\geq$ 0.75 **all five tenants are clamped to +2.0**, yielding an identical* cutoff vector across those settings. The coincident F1 at alpha in {0.50, 0.75, 1.00} therefore reflects **clamping to a common ceiling**, not the gate remaining insensitive to gap position over that range. Accordingly, we scope the robustness claim to the **un-clamped regime alpha in [0.25, 0.50]**, where F1 is $\geq$ 0.968 and the cutoffs still move with alpha. Within that scoped range, at **alpha = 0.50 the label-free calibration matches the per-tenant oracle exactly (F1 0.992)** — placing the cutoff at the midpoint of the gap is, on this corpus, as good as the best per-tenant threshold a labelled set could pick.

We nonetheless **default to the more conservative alpha = 0.25** rather than the F1-maximising 0.50. The reason is the validity threat from Section 6.1: our IN probes are easy passage-derived spans, so the gap measured here is wider and cleaner than it will be for genuine natural-language questions, where in-domain scores are lower and the relevant/irrelevant bands sit closer together. A cutoff that sits only a quarter into the (here generous) gap is the safer choice once that gap narrows on harder queries — it favours recall (grounding the answer) over precision (rejecting to general knowledge) in the borderline cases the micro-benchmark does not stress. We report the full sweep so that a deployment with access to harder labelled queries can revisit this choice.

7. Discussion, Limitations, and Threats to Validity

7.1 What the results do and do not show

The central empirical claim of this paper is deliberately narrow. On a real, deployed, ten-book Arabic+English corpus of 3,739 chunks, an unsupervised, label-free, per-tenant threshold derived purely from the relevance scorer's own score geometry recovers most of the discriminative power that would otherwise require a labeled evaluation set. In the labeled gate-decision benchmark (Exp2), the self-calibrated gate reaches accuracy 0.967, precision 0.938, recall 1.000, and F1 0.968, against the deployable fixed default (the system's previous hand-tuned global cutoff of -5.0) at accuracy 0.792, precision 0.706, recall 1.000, F1 0.828. The improvement is almost entirely in precision: the method raises precision from 0.706 to 0.938 while holding recall at 1.000. Concretely, it removes the off-topic acceptances — passages that score above a too-permissive global cutoff but are not actually relevant — that are the proximate cause of mis-attributed and hallucinated grounded answers, without dropping any genuinely relevant passage on this corpus.

We are equally explicit about what the numbers do *not* show, because the comparison can

be read uncharitably. A *labeled* global cutoff of +3.04 attains F1 1.000 on this corpus, and a *labeled* per-tenant oracle attains F1 0.992. A label-tuned global cutoff is therefore perfect here. This is not evidence against our method; it is the premise of the problem. Obtaining +3.04 requires a labeled relevance set spanning every tenant’s private corpus — exactly the artifact that a private, cold-start, on-device, multi-tenant deployment does not and cannot have, since the corpora are user-owned, never leave the device, and may have zero usage history at the moment a threshold is first needed. The honest, like-for-like comparison is between methods that consume no labels: self-calibrated (F1 0.968) versus the deployable fixed default (F1 0.828). Against the per-tenant oracle (F1 0.992) we claim *near-oracle* performance obtained *without labels* in a setting where the oracle is unobtainable — we explicitly do **not** claim to beat, match, or approach a labeled oracle as if labels were free. This positioning mirrors the training-free, label-free framing of adaptive-gating work such as TARG [42], CRAG’s lightweight retrieval evaluator [46], and the self-reflective relevance decisions of Self-RAG [6] and Adaptive-RAG [20], with the difference that our cutoff is calibrated per tenant and from score geometry alone.

The per-tenant cutoff experiment (Exp1) shows a second, qualitatively different result. Across five heterogeneous tenants the calibrated cross-encoder cutoff spans $[-3.29, -1.39]$ and the cosine cutoff spans $[0.302, 0.409]$; the *pure-Arabic* tenant (T3) receives a higher cutoff in *both* score spaces, and the two *Arabic-containing* tenants (T4, T3) are the two strictest in the cross-encoder space — but the *mixed* tenant (T4) is the cross-encoder maximum yet the cosine minimum, a rank-flip across scorers, so the compensation is real but partial and scorer-dependent rather than a scorer-independent law. The procedure therefore *partially* compensates the relevance model’s per-language scoring offset — it tends to derive a stricter threshold where the multilingual scorer is, on this corpus, systematically less confident — without ever being told which library is Arabic. This is consistent with the general fact that multilingual neural rerankers are imperfectly calibrated [37]; we observe the per-language score offset itself on our own corpus (to our knowledge it has not been separately quantified for this reranker) rather than importing it from prior work. No single global cutoff (-5.0 for the cross-encoder, 0.30 for cosine) fits all five tenants, and the same separating-gap construction works unchanged in both an unbounded-logit space and a bounded-cosine space, which is why we describe the method as scorer-agnostic rather than tied to one ranker family.

7.2 Limitations and threats to validity

We treat the following as genuine threats, not caveats to be minimized.

Micro-benchmark scale and corpus composition. The entire evaluation is a reproducible micro-benchmark run on a single CPU machine over one small, English-dominated, ten-book corpus. It is not a large labeled IR benchmark, and we do not present it as one. With only five tenants and a few hundred queries, the perfect or near-perfect scores of the labeled baselines (global +3.04 at F1 1.000; oracle at F1 0.992) are partly an artifact of how *separable* relevant and non-relevant passages happen to be on a small, topically clean corpus; on a larger, noisier, more topically entangled library the relevant/non-relevant score distributions would overlap more [5, 30], and we would expect every method — including ours — to lose headroom. The reported absolute numbers should be read as evidence of *direction and mechanism* on a real deployment, not as benchmark leaderboard figures.

IN queries are passage-derived spans matched against their own in-pool source, not natural questions. In Exp2, the IN (relevant) queries are 18-word spans of passages from the tenant’s own library, and the source passage each span is cut from **remains in the candidate pool** (it is not held out). This makes the IN side *easy by construction*: a verbatim span shares surface lexical and semantic form with the very passage it was cut from, and that source passage is present to be matched, which inflates retrieval and rerank scores and is the most likely explanation for the perfect recall (1.000) observed across baselines, including

the fixed default. We do not exclude the source passage or do sibling-matching, and we claim neither. Real users ask paraphrased, abbreviated, or multi-hop natural-language questions whose lexical overlap with the source passage is far lower. We therefore expect the *recall* of all configurations — and the absolute margin between methods — to change under natural questions; the recall-favouring default $\alpha = 0.25$ is chosen precisely as a conservative hedge against this, since it sits only a quarter into the separating gap and is more forgiving of harder queries than the higher-precision α values. This is the single threat most likely to move our headline numbers, and we flag it as such.

No end-to-end answer-quality or hallucination study. We measure the *gate decision* (accept/reject), not the *generated answer*. The downstream benefit we attribute to higher precision — fewer mis-sourced, mis-attributed, or hallucinated grounded answers — is *argued* from the precision gain ($0.71 \rightarrow 0.94$), not *measured* on text produced by the local LLM. We have not run a human evaluation of answer faithfulness, a user study, or an automatic hallucination-rate measurement on generated outputs. The causal link from “fewer off-topic acceptances” to “fewer hallucinated answers” is well-motivated by the RAG and corrective-retrieval literature [14, 26, 46] but is, in this paper, an inference rather than a result.

The gate assumes a separating gap. The construction sets $\tau_t = \text{out_hi} + \alpha \cdot (\text{in_lo} - \text{out_hi})$ only when $\text{in_lo} > \text{out_hi}$ — that is, only when the in-domain self-match band (Q25 of S_{in}) lies above the out-domain cross-book band (Q75 of S_{out}). When the scorer cannot separate guaranteed-relevant self-matches from cross-book pairs on a given corpus, no such gap exists, and the method does not fabricate a threshold: it falls back to the safe default cutoff and clamps to the configured range (cross-encoder $[-8, 2]$, cosine $[0, 0.95]$). This is a real coverage limitation — corpora of near-duplicate books, a single dominant topic, or very short documents may fail to produce a gap — but it degrades gracefully to the prior behaviour rather than to a silently wrong cutoff. The requirement of $K \geq 2$ books and $|C_t| \geq 6$ chunks formalizes the minimal structure the construction needs and likewise routes everything smaller to the default.

Anchor construction and determinism. The $S = 8$ pseudo-queries are taken deterministically as the first 12 words of selected chunks, and the out-domain anchor uses one cross-book chunk per query. This makes calibration reproducible and RNG-free, but it also means the threshold is only as representative as those eight anchors; a pathological chunk (boilerplate, a title page, an index) could bias an anchor. We mitigate this with quartile statistics (Q25/Q75) rather than extrema, but we have not stress-tested adversarial or degenerate chunk content, and on multilingual corpora the choice of which books supply anchors interacts with the per-language bias compensation discussed in §7.1.

Single scorer family per space; no cross-scorer transfer study. We demonstrate scorer-agnosticism across two score spaces (a cross-encoder, mmarco-mMiniLMv2-L12-H384-v1, and embedding cosine from paraphrase-multilingual-MiniLM-L12-v2), but we do not study how a τ_t calibrated for one scorer transfers — it does not, and is not meant to: the cache key is (tenant, scorer, corpus-state), and a scorer change invalidates the threshold by design. We have not evaluated late-interaction rerankers [25] or larger cross-encoders, so the $[\text{lo}, \text{hi}]$ clamps are calibrated to the rankers we ship and would need re-derivation for others.

Ablation scope, and a clamping artifact in the plateau. The α ablation (Exp4) holds recall at 1.000 throughout and reports F1 of 0.736, 0.845, 0.968, 0.992, 0.992 for $\alpha \in \{0.00, 0.10, 0.25, 0.50, 0.75, 1.00\}$. We scope the robustness claim to the **un-clamped regime $\alpha \in [0.25, 0.50]$** , where $F1 \geq 0.968$ and at $\alpha = 0.50$ the label-free calibration *matches* the per-tenant oracle (0.992). The apparently flat F1 for $\alpha \geq 0.50$ is partly a **clamping artifact**, not continued insensitivity to gap position: as α grows, $\tau_t = \text{out_hi} + \alpha \cdot (\text{in_lo} - \text{out_hi})$ drives several tenant cutoffs into the cross-encoder clamp ceiling $\text{hi} = +2.0$ — at $\alpha \geq 0.50$ several tenants are clamped, and at $\alpha \geq 0.75$ *all five* are clamped to $+2.0$, producing an identical cutoff vector — so the identical F1 at $\alpha \in \{0.50, 0.75, 1.00\}$ reflects clamping to a common ceiling,

not gap-position robustness over that range. This ablation also inherits every limitation above — in particular the easy-IN-query effect that pins recall at 1.000 — so the flatness of F1 for $\alpha \geq 0.50$ should not be over-read: under harder natural questions the recall ceiling that makes these values coincide would no longer hold, which is exactly why we keep the conservative recall-favouring default at $\alpha = 0.25$ rather than the F1-maximizing $\alpha = 0.50$.

7.3 Ethics, privacy, and fairness

On-device by construction. The system is fully offline and on-device. Ingestion, chunking, hybrid retrieval (bm25s sparse sub-index plus FAISS-HNSW dense index), cross-encoder reranking, threshold calibration, generation (Ollama/llama.cpp), and the conditional cross-lingual translation all run locally; no corpus content, query, embedding, or derived threshold leaves the device, and there is no backend retrieval server. The calibration itself is computed *from local data only* and stores nothing beyond a cached scalar τ_t keyed to the tenant’s own corpus state. This places the work in the on-device, privacy-preserving RAG tradition [35, 40, 44] and means the method’s privacy properties are structural rather than policy-based: there is no telemetry to opt out of, because the data path never reaches a network. Multi-tenant isolation is enforced at every stage by filtering on `user_id`, so one tenant’s content, anchors, and threshold cannot influence another’s results. Critically, the calibration needs *no user feedback, clicks, training signal, or reward* — it is cold-start and works on a brand-new tenant with zero history — which removes a common privacy hazard of adaptive systems that learn from behavioural logs.

Low-resource and Arabic inclusion. The system is bilingual Arabic+English with Iraqi-dialect post-processing, and the calibration’s partial per-language compensation (§7.1) is, in effect, a fairness mechanism: because the multilingual scorer is, on our corpus, less confident on Arabic, a single global cutoff would tend to systematically under-accept Arabic-containing libraries; the per-tenant threshold corrects for this — partially and scorer-dependently — without supervision. This is aligned with the broader effort to serve Arabic and dialectal users with retrieval-augmented systems [1, 3, 7, 33] and to make cross-lingual retrieval viable for low-resource languages [2, 31].

Fairness is related but not measured. We are careful not to overclaim here. Recent work shows that injecting retrieved context can *undermine* fairness in LLMs even for vigilant users [17], and that post-retrieval filtering of biased content matters especially for the small local models we deploy [47]. Our relevance gate is structurally analogous to such a post-retrieval filter — it discards low-relevance passages before generation — but its objective is *relevance*, not *fairness*, and we have not measured bias amplification on generated answers. We therefore present fairness as a related, downstream concern that a relevance gate plausibly helps but does not by itself solve; auditing whether the gate (and the conditional cross-lingual fallback) reduces or merely relocates bias is future work, not a claim of this paper.

8. Conclusion and Future Work

We presented a self-calibrating, per-tenant, label-free relevance gate for the post-retrieval reranking stage of an offline, on-device, multilingual personal RAG library, together with a conditional cross-lingual fallback gated on it. The threshold is derived entirely from the relevance scorer’s own score geometry — an in-domain self-match anchor versus an out-domain cross-book anchor — so it needs no labels, no user feedback, no training, and no evaluation set; it works cold-start on a brand-new tenant, runs on local data only, and is scorer-agnostic across unbounded cross-encoder logits and bounded cosine scores. On a real ten-book Arabic+English deployment it raises gate precision from 0.71 to 0.94 while holding recall at 1.000 (F1 0.828 $\rightarrow$ 0.968) relative to the deployable fixed default, reaching F1 0.968 against a per-tenant oracle’s 0.992 — near-oracle quality in a setting where the oracle is, by privacy and cold-start constraints, unobtainable. As an unsought consequence,

the same procedure partially compensates the scorer’s per-language score offset, assigning the pure-Arabic tenant a stricter cutoff in both score spaces (the compensation is real but partial and scorer-dependent — the mixed Arabic+English tenant rank-flips across the two scorers).

We have been explicit about the boundaries of this evidence: a single-machine micro-benchmark on a small, English-dominated corpus; IN queries that are passage-spans rather than natural questions and therefore likely easy on recall; a downstream benefit argued from precision rather than measured on generated answers; and a method that assumes a separating score gap and defaults safely when none exists. None of these undermine the contribution, but all of them shape what it claims.

The future work that follows directly from these limitations is: (i) a **natural-question benchmark** — replacing passage-span IN queries with paraphrased, abbreviated, and multi-hop user questions to measure how recall and the inter-method margin hold up; (ii) **larger and more genuinely multilingual corpora** with overlapping topics, to test the separating-gap assumption and the bias-compensation effect at scale; (iii) an **end-to-end answer-quality study** — human and automatic measurement of faithfulness, mis-attribution, and hallucination on generated answers, plus a fairness audit linking the gate and cross-lingual fallback to the concerns raised by Hu et al. [17] and Zhang et al. [47]; and (iv) **drift-triggered recalibration** — making threshold invalidation responsive not only to corpus edits but to detected shifts in the score distribution as a tenant’s library and usage evolve. The maktaba-web-local system and the deterministic experiment scripts that produced every number above are reduced to practice and released for reproduction.

Declarations

Competing interests. The author declares no competing interests.

Funding. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Data and Code Availability. The maktaba-web-local system and the deterministic experiment scripts used to produce all reported results are released publicly for reproduction. The evaluation corpus consists of user-owned books; because the system is on-device and privacy-preserving, the released artifact provides the scripts and configuration needed to reproduce the methodology on a comparable corpus rather than redistributing private library content. The public repository is at <https://github.com/aymnkadymy-hub/maktaba-web-local>, archived at Zenodo (concept DOI, all versions): <https://doi.org/10.5281/zenodo.20688577>.

References

- [1] Abdul-Mageed, Muhammad; Zhang, Chiyu; Elmadany, AbdelRahim; Bouamor, Houda; Habash, Nizar (2021). NADI 2021: The Second Nuanced Arabic Dialect Identification Shared Task. *Proceedings of the Sixth Arabic Natural Language Processing Workshop (WANLP 2021)*. <https://aclanthology.org/2021.wanlp-1.28/>
- [2] Adeyemi, M., Oladipo, A., Pradeep, R., Lin, J. (2024). Zero-Shot Cross-Lingual Reranking with Large Language Models for Low-Resource Languages. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*, Vol. 2: Short Papers, pp. 650-656. <https://doi.org/10.18653/v1/2024.acl-short.59>
- [3] Antoun, Wissam; Baly, Fady; Hajj, Hazem (2020). AraBERT: Transformer-based Model for Arabic Language Understanding. *Proceedings of the 4th Workshop on Open-Source Arabic*

Corpora and Processing Tools (OSACT4), with a Shared Task on Offensive Language Detection (at LREC 2020). <https://aclanthology.org/2020.osact-1.2/>

[4] Arampatzis, A., Kamps, J., Robertson, S. (2009). Where to stop reading a ranked list? Threshold optimization using truncated score distributions. *SIGIR 2009 (32nd ACM SIGIR Conference)*. <https://doi.org/10.1145/1571941.1572031>

[5] Arampatzis, A., Robertson, S. (2011). Modeling score distributions in information retrieval. *Information Retrieval (journal)*, 14(1). <https://doi.org/10.1007/s10791-010-9145-5>

[6] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *ICLR 2024 (Oral)*. <https://arxiv.org/abs/2310.11511>

[7] Bechiri, El Batoul; Lanasri, Dihia (2026). DziriBOT: RAG Based Intelligent Conversational Agent for Algerian Arabic Dialect. *arXiv preprint*. <https://arxiv.org/abs/2602.02270>

[8] Bonifacio, L., Jeronymo, V., Abonizio, H. Q., Campiotti, I., Fadaee, M., Lotufo, R., Nogueira, R. (2021). mMARCO: A Multilingual Version of the MS MARCO Passage Ranking Dataset. *arXiv (preprint)*. <https://doi.org/10.48550/arXiv.2108.13897>

[9] Ch, Dhawaleswar Rao; Saha, Sujun Kumar (2020). Automatic Multiple Choice Question Generation From Text: A Survey. *IEEE Transactions on Learning Technologies*, vol. 13, no. 1, pp. 14-25. <https://doi.org/10.1109/TLT.2018.2889100>

[10] Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods. *SIGIR 2009 (32nd ACM SIGIR Conference on Research and Development in Information Retrieval)*, pp. 758-759. <https://doi.org/10.1145/1571941.1572114>

[11] da Silva de Oliveira, M. V., de Andrade Silva, J., de Lima Fontao, A. (2025). Fairness Testing in Retrieval-Augmented Generation: How Small Perturbations Reveal Bias in Small Language Models. *arXiv (cs.SE); to appear at SBQS 2025 (Brazilian Symposium on Software Quality)*. <https://doi.org/10.48550/arXiv.2509.26584>

[12] Dunlosky, John; Rawson, Katherine A.; Marsh, Elizabeth J.; Nathan, Mitchell J.; Willingham, Daniel T. (2013). Improving Students' Learning With Effective Learning Techniques: Promising Directions From Cognitive and Educational Psychology. *Psychological Science in the Public Interest*, vol. 14, no. 1, pp. 4-58. <https://doi.org/10.1177/1529100612453266>

[13] Gao, L., Ma, X., Lin, J., Callan, J. (2023a). Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE). *ACL 2023 (Proceedings of the 61st Annual Meeting of the ACL, Volume 1: Long Papers)*, pp. 1762-1777. <https://doi.org/10.18653/v1/2023.acl-long.99>

[14] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., Wang, H. (2023b). Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv (preprint)*. <https://doi.org/10.48550/arXiv.2312.10997>

[15] Gollapudi, S., Karia, N., Sivashankar, V., Krishnaswamy, R., Begwani, N., Raz, S., Lin, Y., Zhang, Y., Mahapatro, N., Srinivasan, P., Singh, A., Simhadri, H. V. (2023). Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. *Proceedings of the ACM Web Conference 2023 (WWW '23)*, pp. 3406-3416. <https://doi.org/10.1145/3543507.3583552>

[16] Guu, K., Lee, K., Tung, Z., Pasupat, P., Chang, M. (2020). REALM: Retrieval-Augmented Language Model Pre-Training. *ICML 2020 (PMLR vol. 119)*. <https://arxiv.org/abs/2002.08909>

[17] Hu, M., Wu, H., Guan, Z., Zhu, R., Guo, D., Qi, D., Li, S. (2024). No Free Lunch: Retrieval-Augmented Generation Undermines Fairness in LLMs, Even for Vigilant Users. *arXiv (cs.CL/cs.CY)*. <https://doi.org/10.48550/arXiv.2410.07589>

- [18] Izacard, G., Grave, E. (2021). Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. *EACL 2021 (16th Conf. of the European Chapter of the ACL: Main Volume)*. <https://doi.org/10.18653/v1/2021.eacl-main.74>
- [19] Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., Grave, E. (2022). Unsupervised Dense Information Retrieval with Contrastive Learning. *Transactions on Machine Learning Research (TMLR)*. <https://arxiv.org/abs/2112.09118>
- [20] Jeong, S., Baek, J., Cho, S., Hwang, S. J., & Park, J. (2024). Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. *NAACL 2024 (Long Papers)*, pp. 7036-7050. <https://doi.org/10.18653/v1/2024.naacl-long.389>
- [21] Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., & Neubig, G. (2023). Active Retrieval Augmented Generation. *EMNLP 2023 (Main)*, pp. 7969-7992. <https://doi.org/10.18653/v1/2023.emnlp-main.495>
- [22] Jin, Y., Wu, Y., Hu, W., Maggs, B. M., Zhang, X., Zhuo, D. (2024). Curator: Efficient Indexing for Multi-Tenant Vector Databases. *arXiv preprint (cs.DB / cs.IR)*. <https://doi.org/10.48550/arXiv.2401.07119>
- [23] Johnson, J., Douze, M., and Jegou, H. (2021). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535-547 (*IEEE early access / accepted 2019*). <https://doi.org/10.1109/TBDATA.2019.2921572>
- [24] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *EMNLP 2020 (Conference on Empirical Methods in Natural Language Processing)*, pp. 6769-6781. <https://doi.org/10.18653/v1/2020.emnlp-main.550>
- [25] Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. *SIGIR 2020 (43rd ACM SIGIR Conference on Research and Development in Information Retrieval)*, pp. 39-48. <https://doi.org/10.1145/3397271.3401075>
- [26] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020 (Advances in Neural Information Processing Systems 33)*. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [27] Lin, J., Alfonso-Hermelo, D., Jeronymo, V., Kamaloo, E., Lassance, C., Nogueira, R., Ogundepo, O., Rezagholizadeh, M., Thakur, N., Yang, J.-H., Zhang, X. (2023). Simple Yet Effective Neural Ranking and Reranking Baselines for Cross-Lingual Information Retrieval. *arXiv (cs.IR) / TREC 2022 NeuCLIR Track context*. <https://doi.org/10.48550/arXiv.2304.01019>
- [28] Lu, X. H. (Xing Han Lù) (2024). BM25S: Orders of Magnitude Faster Lexical Search via Eager Sparse Scoring. *arXiv preprint arXiv:2407.03618 (technical report)*. <https://doi.org/10.48550/arXiv.2407.03618>
- [29] Malkov, Yu. A., and Yashunin, D. A. (2020). Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 42, no. 4, pp. 824-836 (*IEEE early access / accepted 2018*). <https://doi.org/10.1109/TPAMI.2018.2889473>
- [30] Manmatha, R., Rath, T. M., Feng, F. (2001). Modeling score distributions for combining the outputs of search engines. *SIGIR 2001 (24th ACM SIGIR Conference)*. <https://doi.org/10.1145/383952.384005>
- [31] Nie, J.-Y. (2010). Cross-Language Information Retrieval. *Synthesis Lectures on Human Language Technologies (Vol. 3, No. 1)*, Morgan & Claypool, pp. 1-125.

<https://doi.org/10.2200/S00266ED1V01Y201005HLT008>

- [32] Nogueira, R., Cho, K. (2019). Passage Re-ranking with BERT. *arXiv (preprint)*. <https://doi.org/10.48550/arXiv.1901.04085>
- [33] Obeid, Ossama; Zalmout, Nasser; Khalifa, Salam; Taji, Dima; Oudah, Mai; Alhafni, Bashar; Inoue, Go; Eryani, Fadhl; Erdmann, Alexander; Habash, Nizar (2020). CAMEL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing. *Proceedings of the Twelfth Language Resources and Evaluation Conference (LREC 2020)*. <https://aclanthology.org/2020.lrec-1.868/>
- [34] Otsu, N. (1979). A Threshold Selection Method from Gray-Level Histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1). <https://doi.org/10.1109/TSMC.1979.4310076>
- [35] Park, Taehwan; Lee, Geonho; Kim, Min-Soo (2025). MobileRAG: A Fast, Memory-Efficient, and Energy-Efficient Method for On-Device RAG. *arXiv preprint (cs.DB)*. <https://doi.org/10.48550/arXiv.2507.01079>
- [36] Patel, L., Kraft, P., Guestrin, C., Zaharia, M. (2024). ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proceedings of the ACM on Management of Data (PACMOD)*, Vol. 2 No. 3 (SIGMOD 2024). <https://doi.org/10.1145/3654923>
- [37] Penha, G., Hauff, C. (2021). On the Calibration and Uncertainty of Neural Learning to Rank Models for Conversational Search. *EACL 2021 (Main Volume)*. <https://doi.org/10.18653/v1/2021.eacl-main.12>
- [38] Robertson, S. E., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389. <https://doi.org/10.1561/15000000019>
- [39] Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., Manning, C. D. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. *ICLR 2024 (International Conference on Learning Representations)*. <https://arxiv.org/abs/2401.18059>
- [40] Seemakhupt, Korakit; Liu, Sihang; Khan, Samira (2024). EdgeRAG: Online-Indexed RAG for Edge Devices. *arXiv preprint (cs.LG)*. <https://doi.org/10.48550/arXiv.2412.21023>
- [41] Tabibian, Behzad; Upadhyay, Utkarsh; De, Abir; Zarezade, Ali; Schölkopf, Bernhard; Gomez-Rodriguez, Manuel (2019). Enhancing human learning via spaced repetition optimization. *Proceedings of the National Academy of Sciences (PNAS)*, vol. 116, no. 10, pp. 3988-3993. <https://doi.org/10.1073/pnas.1815156116>
- [42] Wang, Y., Wei, L., & Ling, H. (2025). Retrieval as a Decision: Training-Free Adaptive Gating for Efficient RAG (TARG). *arXiv preprint (2511.09803)*. <https://doi.org/10.48550/arXiv.2511.09803>
- [43] Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., Zhou, M. (2020). MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. *NeurIPS 2020 (Advances in Neural Information Processing Systems 33)*. <https://proceedings.neurips.cc/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [44] Wang, Zijie J.; Chau, Duen Horng (2024). MeMemo: On-device Retrieval Augmentation for Private and Personalized Text Generation. *SIGIR 2024 (Proc. 47th Int. ACM SIGIR Conf. on Research and Development in Information Retrieval)*, pp. 2765-2770. <https://doi.org/10.1145/3626772.3657662>
- [45] Woźniak, Piotr A.; Gorzelańczyk, Edward J. (1994). Optimization of repetition spacing in the practice of learning. *Acta Neurobiologiae Experimentalis*, vol. 54, no. 1, pp. 59-62. <https://pubmed.ncbi.nlm.nih.gov/8023714/>

- [46] Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). Corrective Retrieval Augmented Generation. *arXiv preprint (2401.15884)*. <https://doi.org/10.48550/arXiv.2401.15884>
- [47] Zhang, Z., Li, N., Liu, Q., Li, R., Gao, W., Mao, Q., Huang, Z., Yu, B., Tao, D. (2025). The Other Side of the Coin: Exploring Fairness in Retrieval-Augmented Generation. *arXiv (cs.CL)*. <https://doi.org/10.48550/arXiv.2504.12323>